

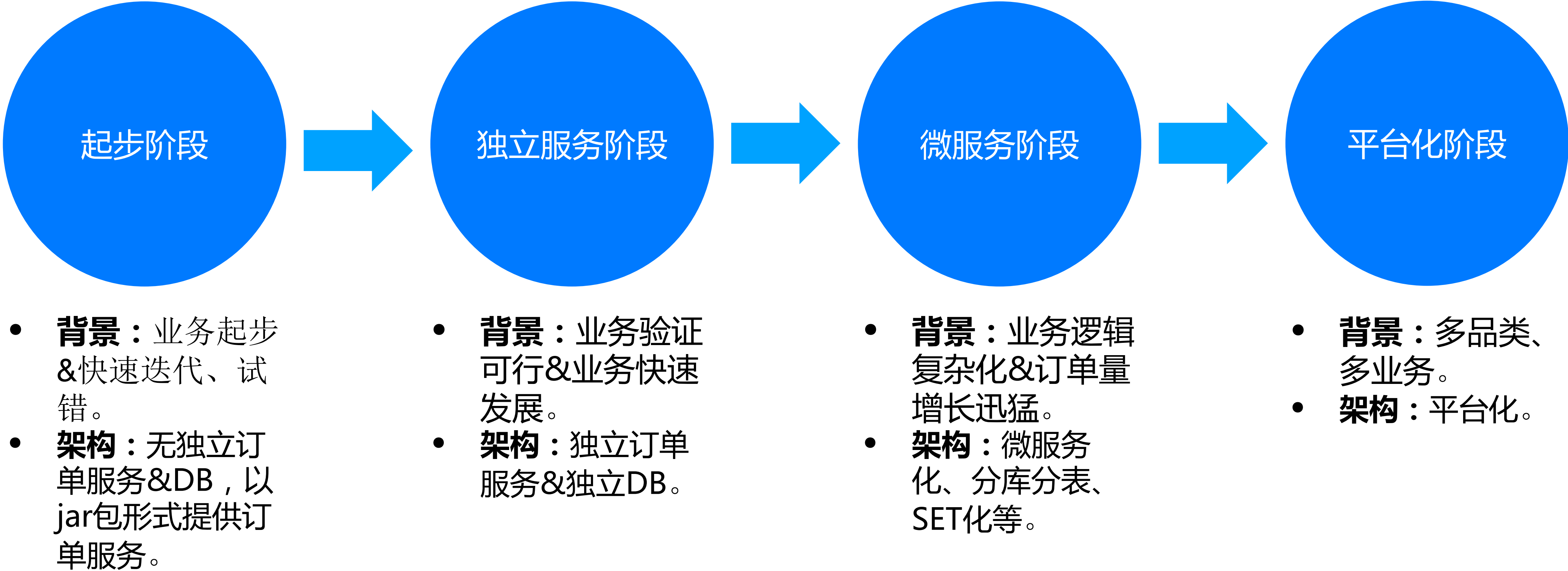
订单平台化建设实践

何轶 2019-12-10

目录

- 订单系统演进过程
- 订单平台化整体架构
- 相关挑战点及解决方案
- 订单平台化架构总结及收益
- 新业务接入案例
- 后续规划

订单系统演进过程

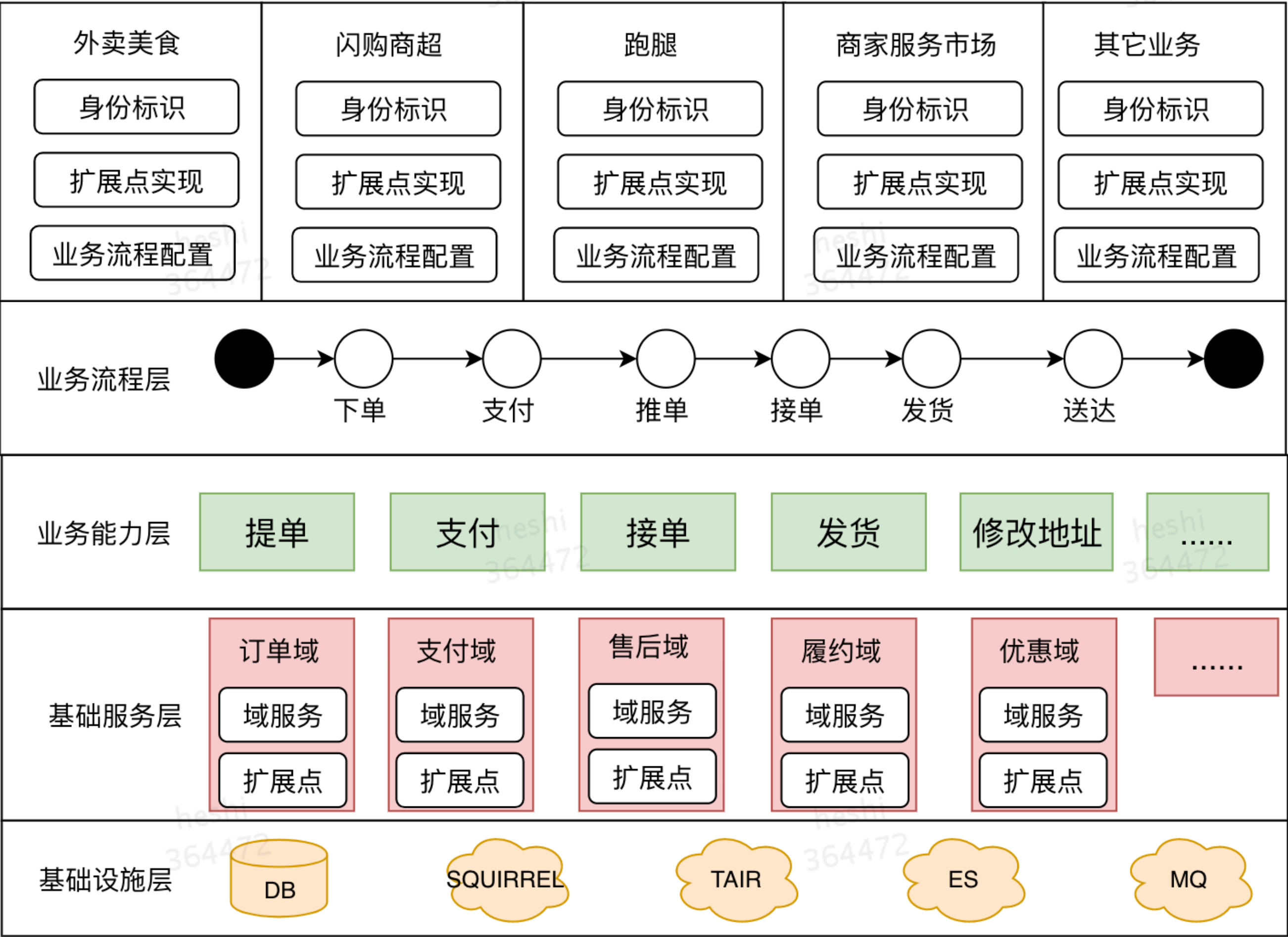


订单平台化整体架构

目标：构建一套通用、可配置、易扩展的订单平台，统一支持多业务、多品类的订单交易。

订单平台化架构：

- ✓ 基础设施层：提供存储、消息、RPC等中间件。
- ✓ 基础服务层：按域组织的基础服务，域服务内针对不同业务的差异提供扩展点。
- ✓ 业务能力层：串联不同域服务形成可供外部使用的业务能力，比如下单、支付等。
- ✓ 业务流程层：对业务能力进行编排，形成订单交易流程，完成订单交易过程。
- ✓ 业务层：指定业务身份、扩展点实现及业务流程配置等，实现不同业务差异。



订单交易平台架构图

问题：多业务、多品类交易功能如何复用、差异流程如何支持？如何避免相互影响？

核心挑战：

- ✓ 通用性&扩展性：
 - 业务身份定义：如何区分不同品类、不同业务。
 - 业务模型扩展：如何支持业务交易模型差异。
 - 业务功能通用&扩展：如何统一支持不同业务交易功能及其差异逻辑。
 - 业务流程扩展：如何支持不同业务差异交易流程。
- ✓ 稳定性：
 - 同一系统支持多业务交易，存在相互影响隐患。
 - 线上问题如何快速定位到问题业务。

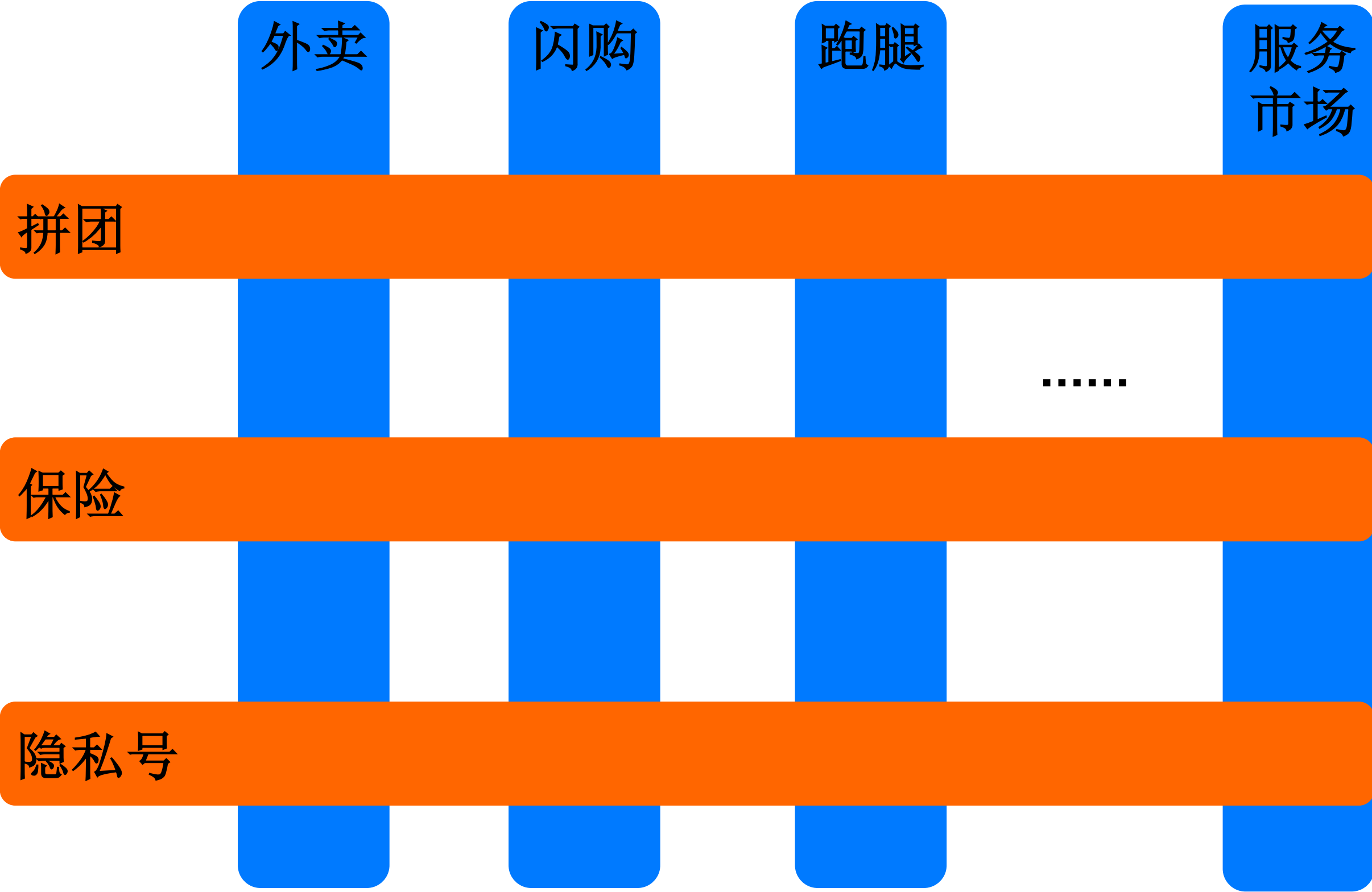


解决方案-业务身份定义

挑战：如何定义一套合理的业务身份，方便区分不同业务？

解决方案：

- ✓ 业务身份：某个独立的业务线或者产品定义为一个业务身份，例如外卖、跑腿、会员等。业务身份间的配置规则完全独立，不相互影响。
- ✓ 业务标识：业务线的一些共性玩法，如拼团等。以及业务线的共用功能，如保险、隐私号等。一个业务标识可以被多个业务使用，一个业务也可以使用多个业务标识。

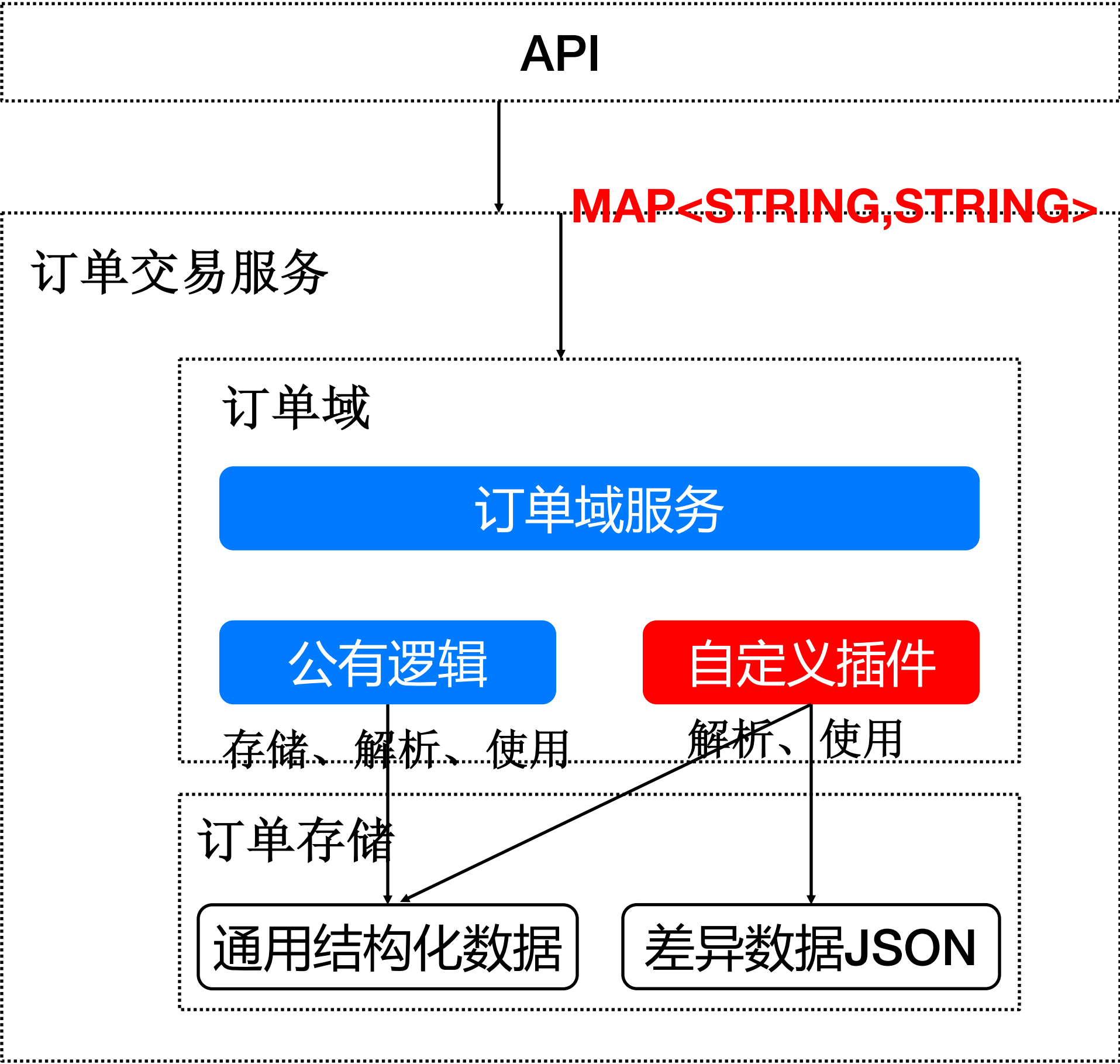


解决方案-业务模型扩展

挑战：多业务的差异模型（例如订单拼团信息等）如何支持？

解决方案：

- ✓ 业务共有信息：业务共有信息进行固化，包括订单基础信息、买家信息、卖家信息等，方便处理。
- ✓ 业务差异信息：服务间传递时，通过MAP结构传递；内部存储为JSON；后续处理时通过业务自定义逻辑处理，与公有逻辑解耦。

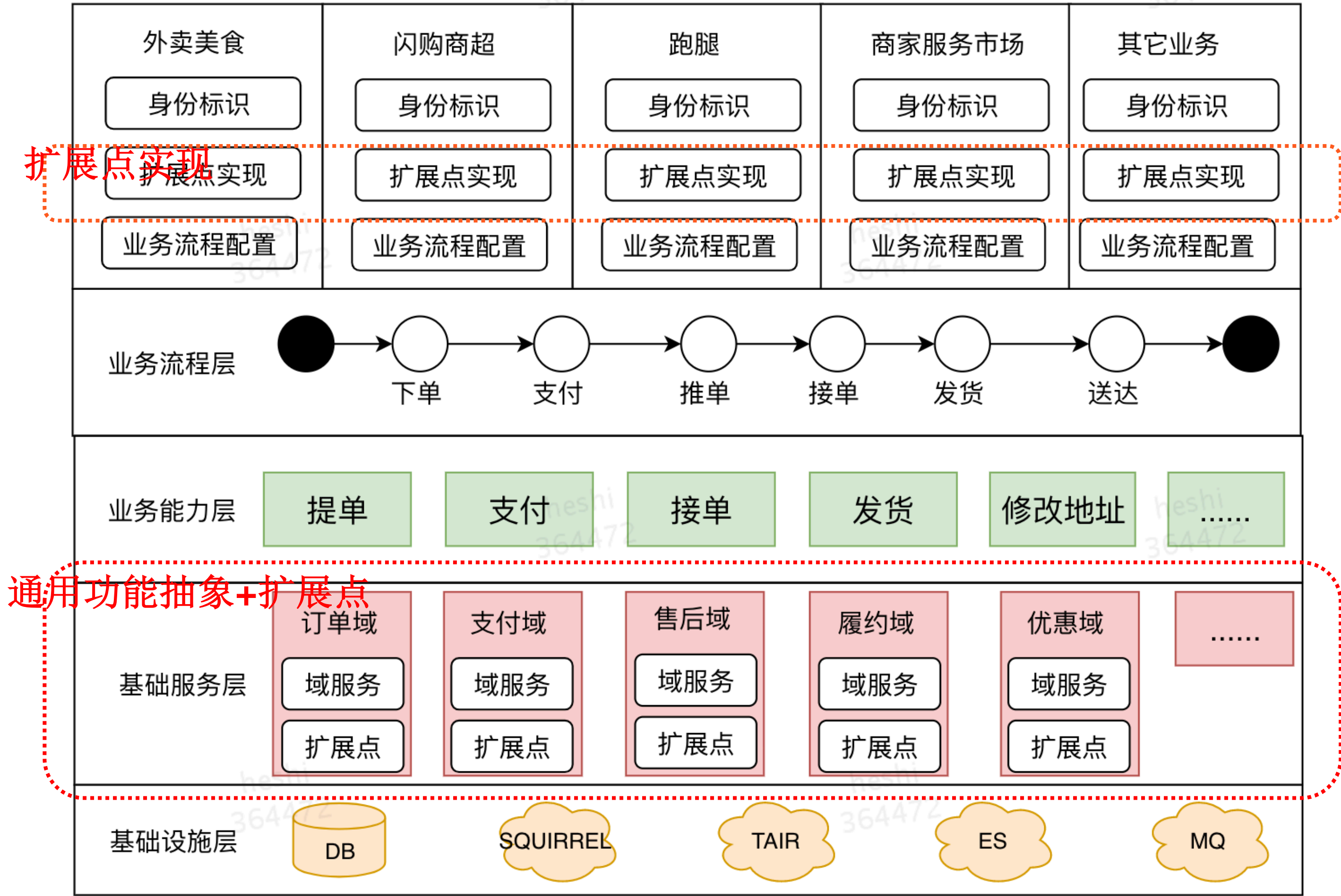


解决方案-业务功能通用&扩展

问题：订单功能如何通用化从而支持多业务，以及多业务差异如何处理？

解决方案：

- ✓ 功能通用化：结合业界调研及对业务理解，对功能及流程进行统一抽象，形成业务通用流程&功能，方便业务复用。
- ✓ 差异逻辑处理：在通用流程中针对可能存在差异的节点，预留扩展点，方便外部扩展实现。



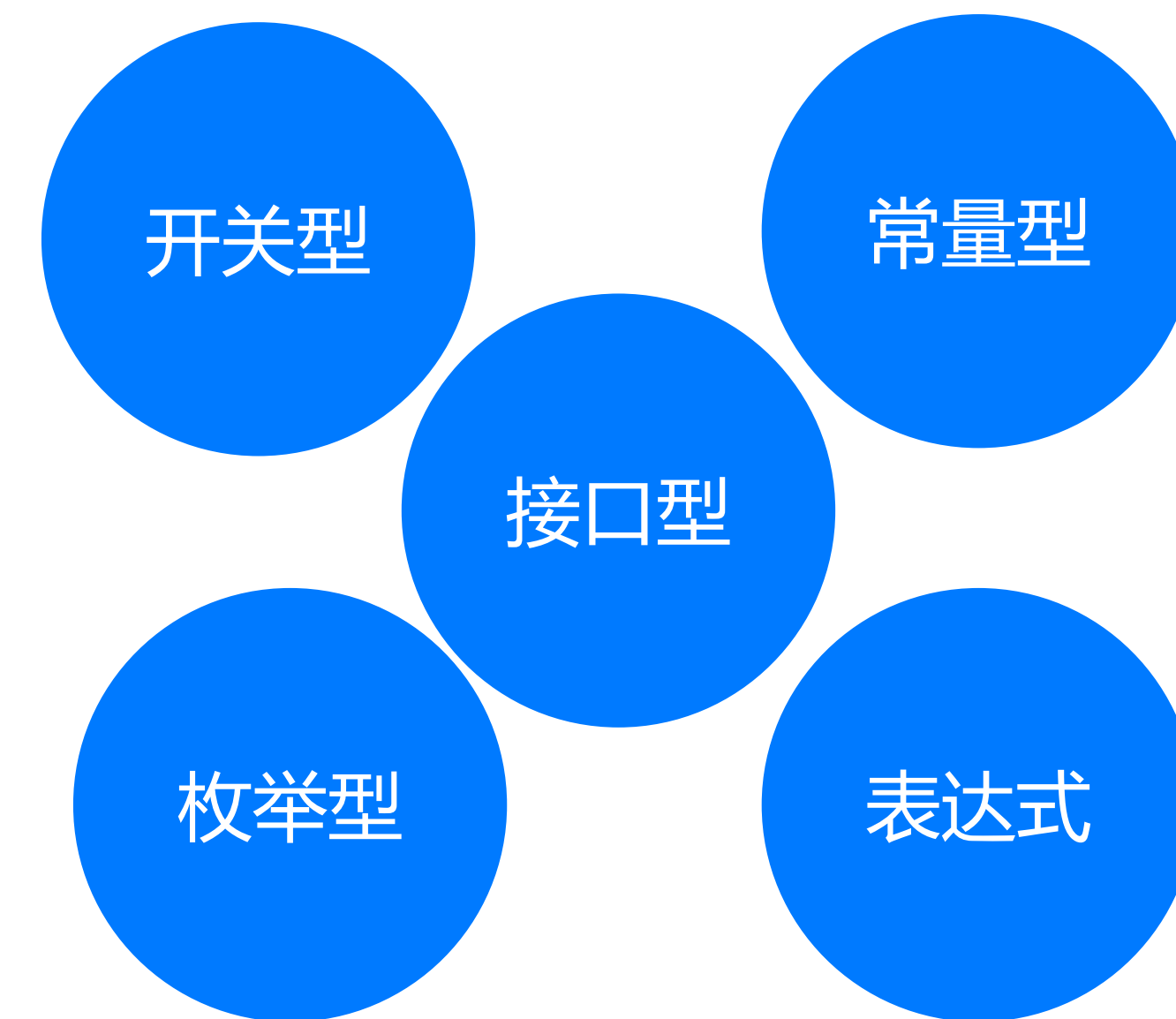
订单交易平台架构图

解决方案-业务功能通用&扩展-扩展点定义

挑战：不同业务差异逻辑各异，扩展点如何定义才能满足需求？

解决方案：

- ✓ 配置型扩展点
 - ✓ 开关型扩展点：定义某个逻辑是否生效。
 - ✓ 常量型扩展点：用于加载文案、帐号、过期时间等业务间存在差异的信息。
 - ✓ 枚举型扩展点：预置多种实现/策略供业务选择，用枚举值表示一类实现/策略。
 - ✓ 表达式扩展点：用于简单的逻辑处理，如条件判断、文案处理等。
- ✓ 接口型扩展点
 - ✓ 接口型扩展点：定义一个接口，外部实现接口处理差异逻辑。

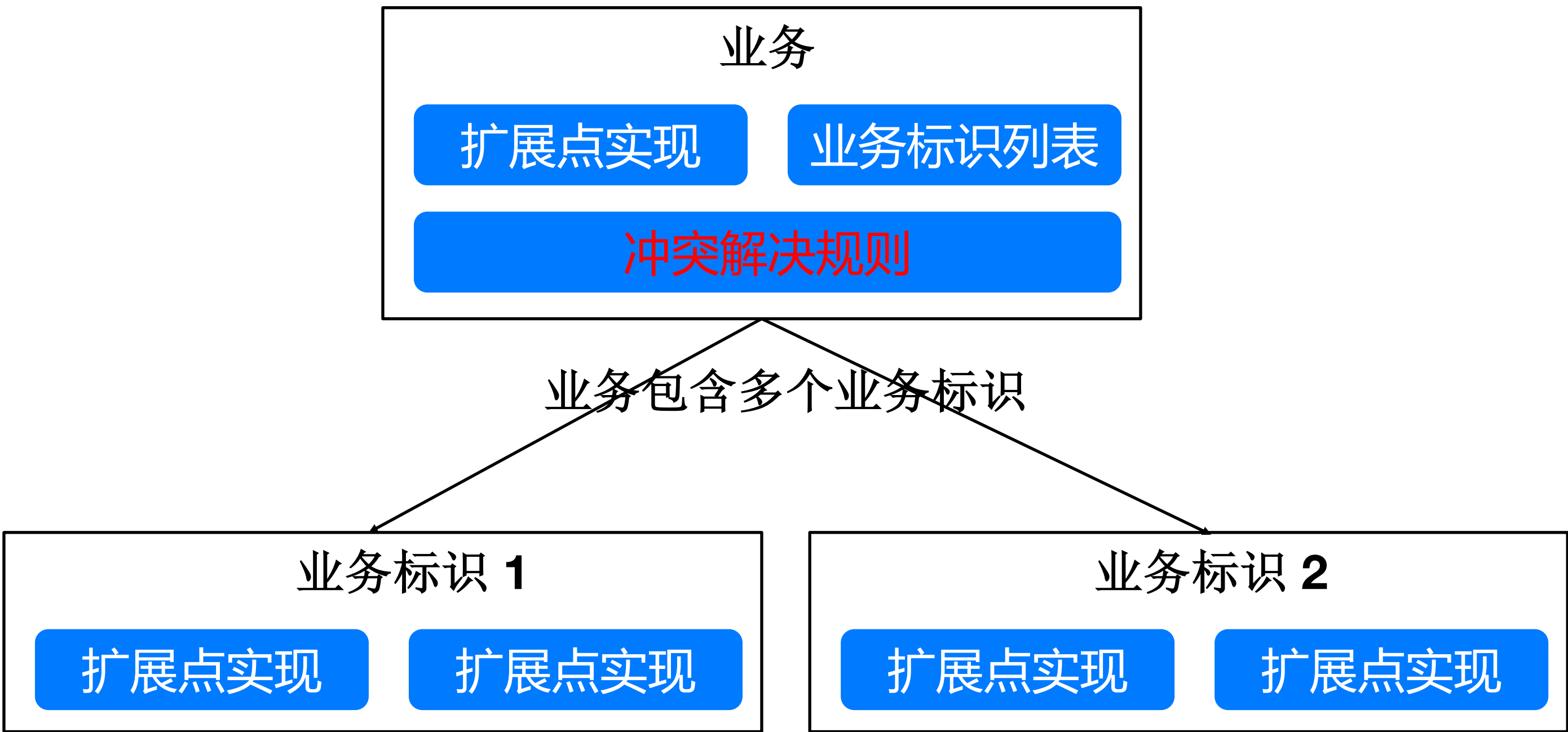


解决方案-业务功能通用&扩展-冲突解决

挑战：在身份定义中，一个业务可以同时使用多个标识，每个业务标识对应一套配置，配置发生冲突时，如何处理呢？

解决方案：业务定义包含的业务标识及冲突解决规则。

- ✓ 互斥型规则：定义优先使用哪个配置。
- ✓ 共享型规则：所有配置共同生效。



解决方案-业务功能通用&扩展-限额功能实例

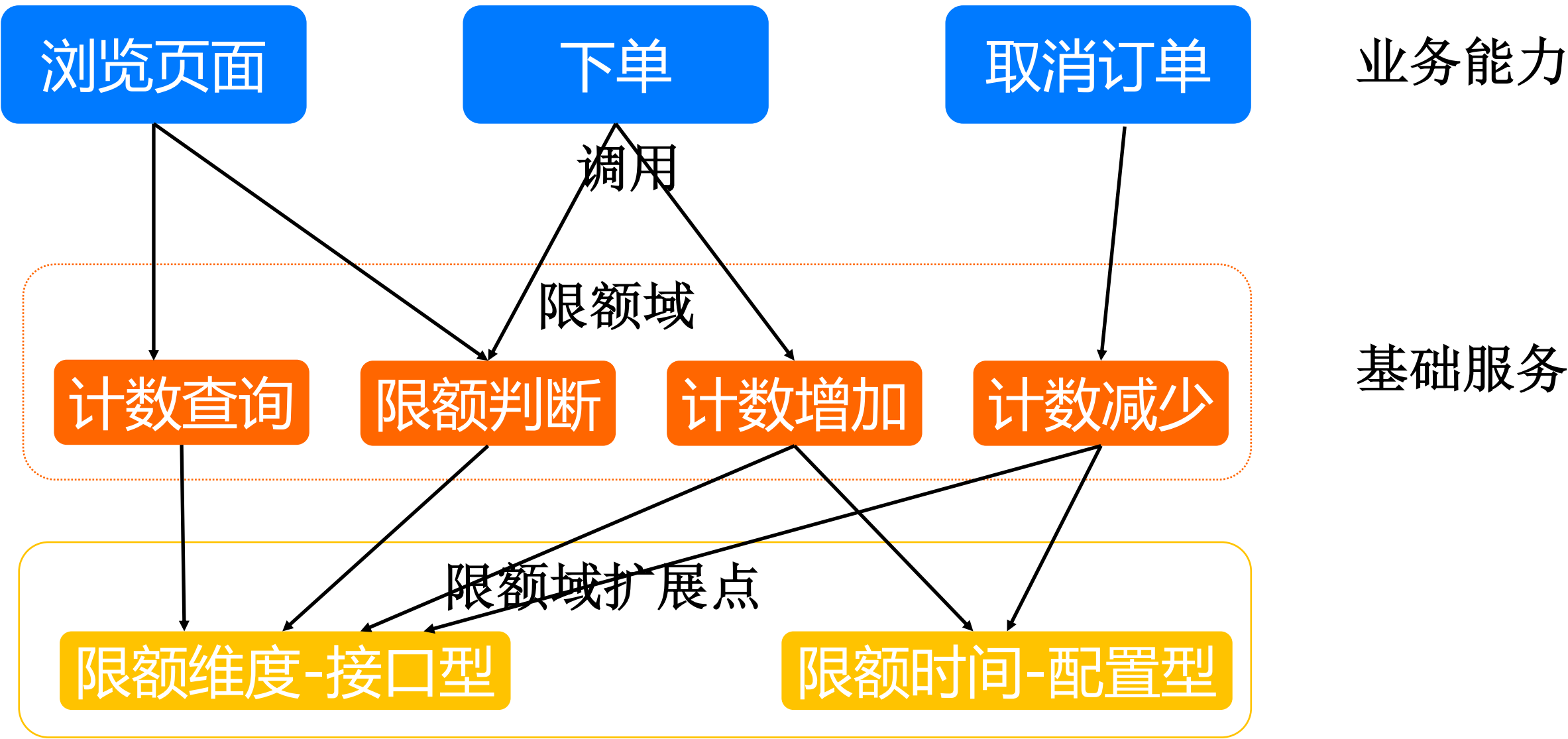
需求描述：用户在购买时，由于优惠等原因，需要限制其购买上限。不同业务的限额维度、时间均不一样，比如限制用户购买次数、限制用户+SKU的购买次数等。

解决方案：

- ✓ 功能抽象：将限额相关功能聚合为限额域，抽象限额域提供限额判断、计数增加、计数减少、计数查询服务。
- ✓ 扩展点：将限额判断、计数增加、计数减少、计数查询四个服务中可变的限额维度、限额时间开放为扩展点，供外部扩展。

收益：

- ✓ 研发成本：限额需求可通过简单配置快速实现，避免了需求设计、评审、开发、联调、测试过程的研发成本。
- ✓ 稳定性方面：超卖、高并发处理等复杂逻辑直接复用，避免研发BUG，有效提升稳定性。



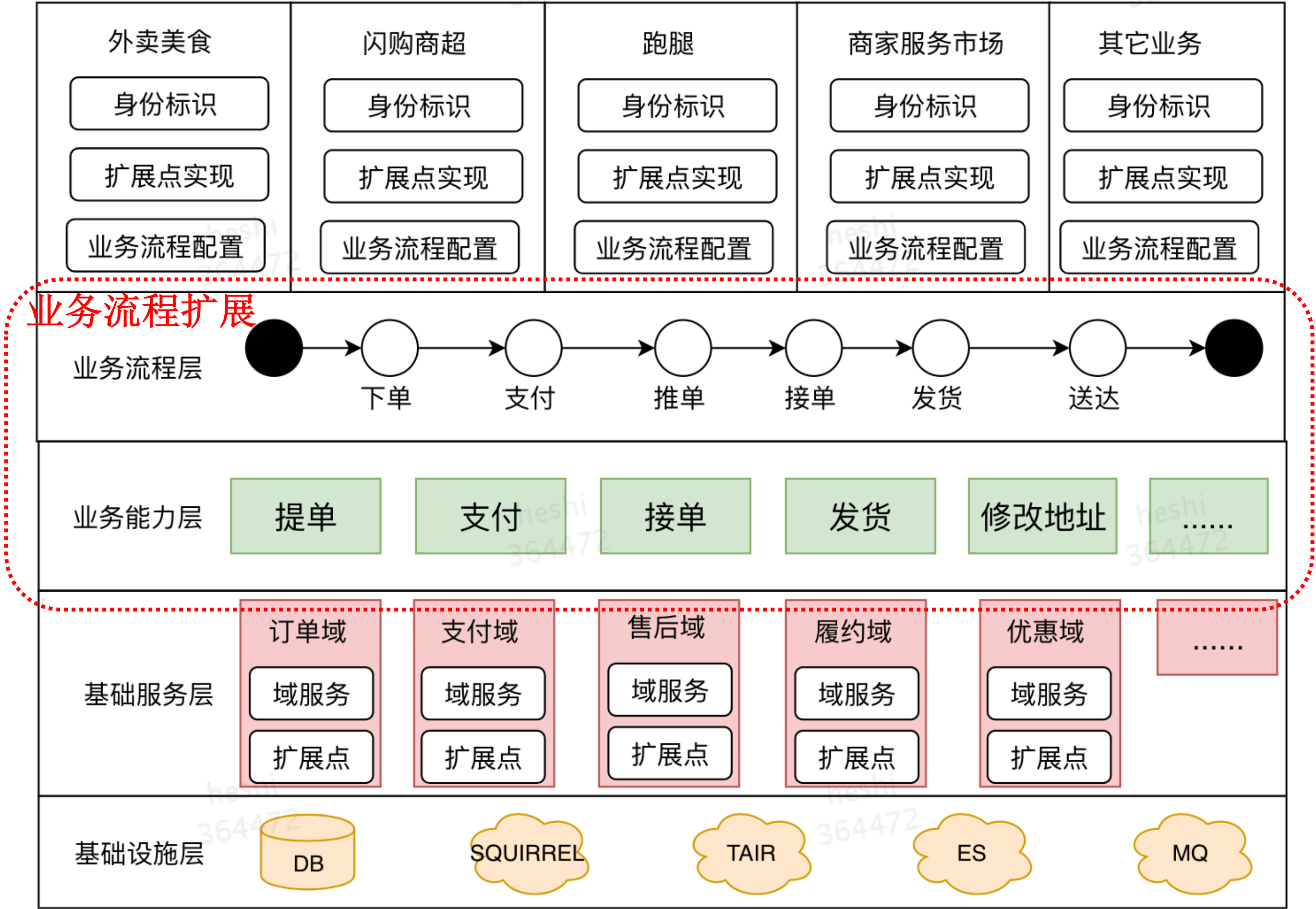
解决方案-业务流程扩展

挑战：不同业务差异性流程如何支持？如何支持业务流程变更及扩展呢？

编排范围：架构中有三层包含流程执行，是否三层都需要支持流程编排呢？

- ✓ 基础服务层：领域功能内的服务执行流程。
- ✓ 业务能力层：串联各个领域服务形成产品功能，比如提单。
- ✓ 业务流程层：编排各个业务能力节点形成业务流程。

编排范围结论：综合业务扩展性需求及架构复杂性，业务流程编排支持业务能力层及业务流程层。



订单交易平台架构图

解决方案-业务流程扩展-流程引擎选型

开源方案：

- ✓ 优势：开源软件如JBPM、Activiti等，支持的功能丰富，比如高级的多分支合并等。同时，可以直接使用，无开发成本。
- ✓ 劣势：功能复杂度高，与现有系统整合成本高且会带来一定的稳定性风险。
- ✓ 适用场景：传统的工作审批流等。

VS

自研方案：

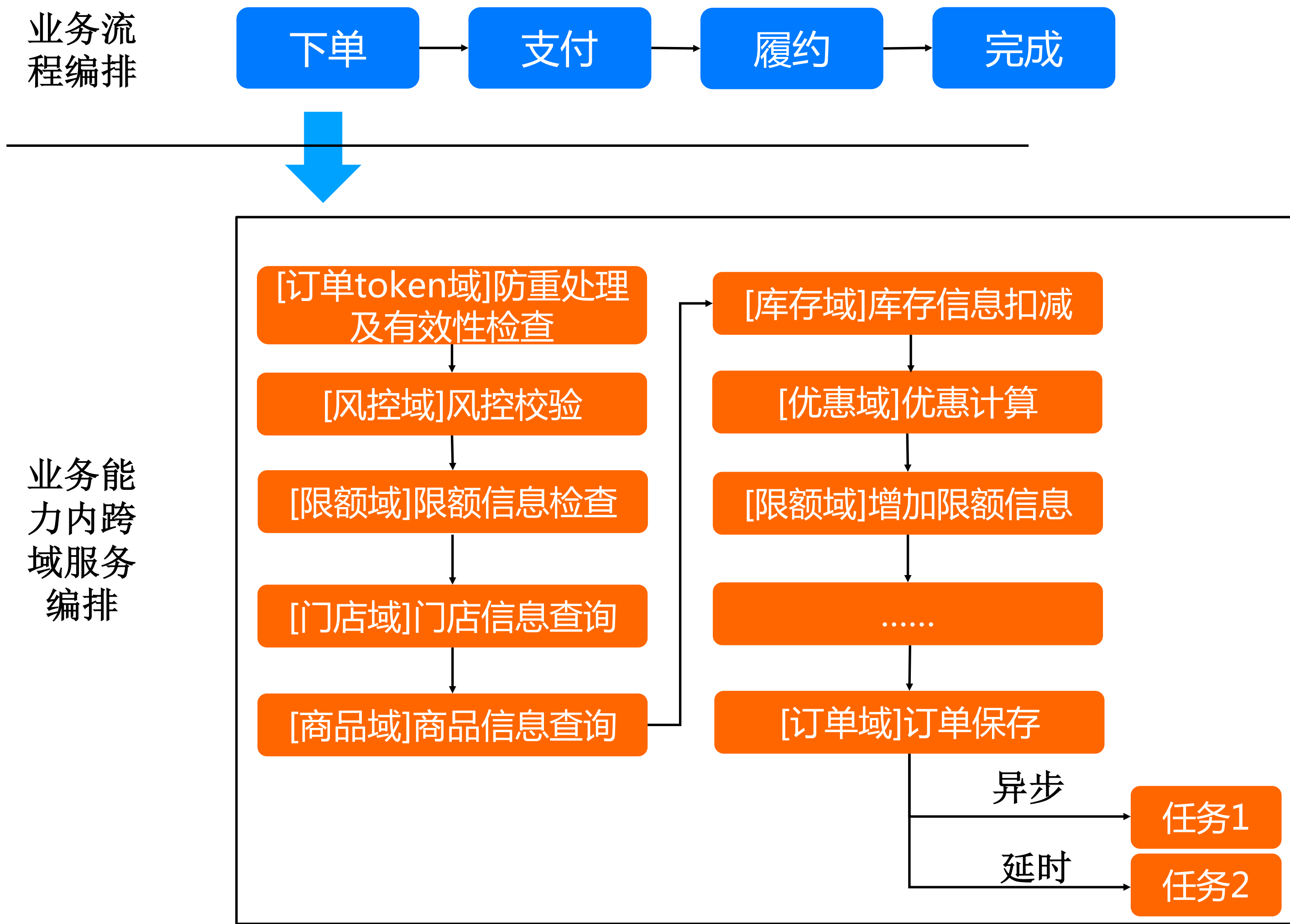
- ✓ 优势：功能可结合交易领域编排要求定制，方案复杂度&风险可控，与现有系统不存在整合成本。
- ✓ 劣势：存在一定的研发成本。
- ✓ 适用场景：特定领域的定制流程引擎。

选型结论：交易流程编排并不需要如此复杂的功能，同时考虑到与现有系统整合成本、后续迭代可控性及稳定性风险，最终选择自研方案，建立一套交易领域内的流程编排引擎。

解决方案-业务流程扩展-流程引擎模型

流程引擎模型：

- ✓ 业务流程编排：支持对业务能力进行编排，形成业务流程，编排的元素为业务能力节点。
- ✓ 跨域服务编排：支持对业务流程编排中的业务能力内部执行流程进行编排，形成业务能力，如下单、支付等，编排的元素为领域服务。



流程引擎实现：

- ✓ 模型表示：类似开源流程引擎，通过XML来表示引擎模型，XML中的元素对应执行节点、依赖关系、执行流等。
- ✓ 模型执行：系统自定义流程引擎解析不同业务的XML后，缓存流程定义。具体执行时，通过反射、表达式计算等方式进行流程执行。

```
<stage id="stage5">
  <dependencies>
    <dependency dependsOn="stage4" condition="#order.is_comment == 1" />
  </dependencies>
  <prechecker type="local" bean="demoStagePreChecker" />
  <statuses main="10" logistics="1" ext="5" />
  <beforeStatusesChange>
    <tasks>
      <task id="stage5_task1" type="local">
        <call bean="localCallSampleBean" method="method">
          <param>#order.wm_vp_order_view</param>
          <param>#testParam</param>
          <param>#order.wm_vp_order_view</param>
        </call>
      </task>
    </tasks>
  </beforeStatusesChange>
  <returns>
    <return key="return1">#stage5_task1Key</return>
    <return key="return2">#testParam</return>
  </returns>
</stage>

<stage id="cancel">
  <afterStatusesChange>
    <tasks>
      <task id="cancel_task1" type="local">
        <call bean="localCallSampleBean" method="method">
          <param>#order.wm_vp_order_view</param>
          <param>'cancel_task1Key'</param>
          <param>#order.wm_vp_order_view</param>
        </call>
      </task>
    </tasks>
  </afterStatusesChange>
</stage>
<flows...>
```

解决方案-稳定性-业务间相互影响

挑战：同一系统支持多业务，存在相互影响隐患？

解决方案：

- ✓ 逻辑隔离：通过插件进行代码隔离。
- ✓ 部署隔离：不同业务部署到不同集群中，进行资源部署隔离。
- ✓ 数据隔离：不同业务读写不同存储，进行数据隔离。

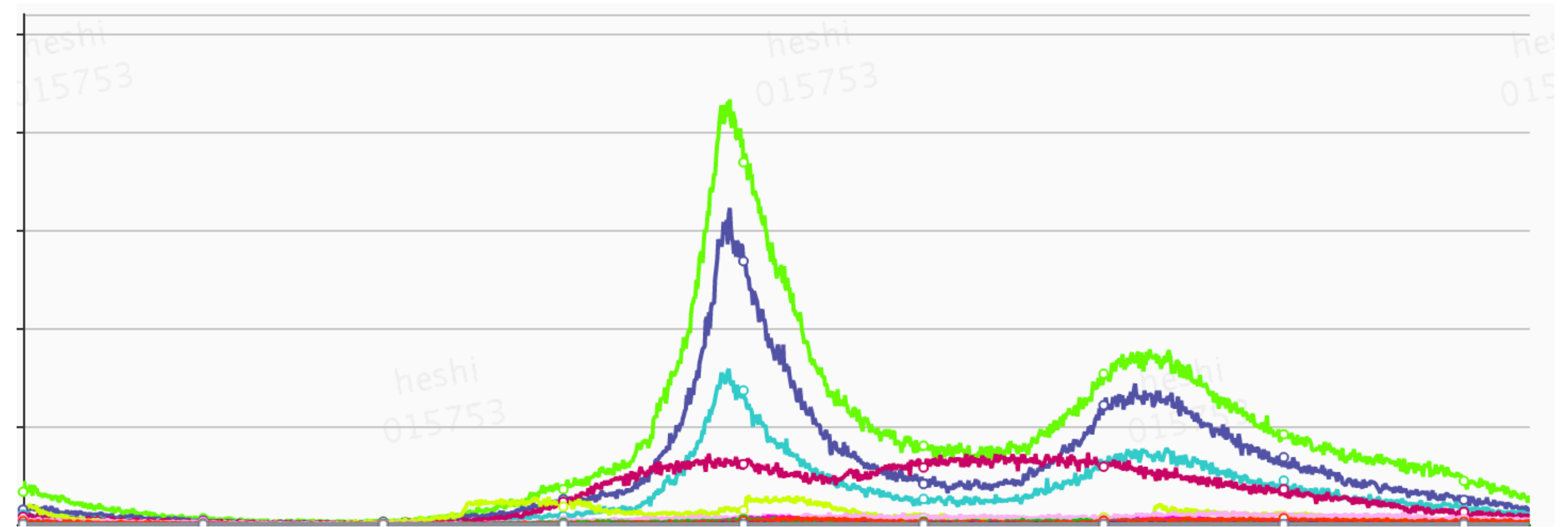


解决方案-稳定性-多业务监控

问题：出现问题，如何快速定位具体是哪个业务问题？

解决方案：监控指标增加业务维度，相关监控指标包括：

- ✓ 系统指标：CPU、NET等；
- ✓ 应用指标：异常日志、性能、调用量等；
- ✓ 业务指标：下单数、支付数、取消数等；



订单交易分业务监控图

订单平台化架构总结

订单平台化架构总结：

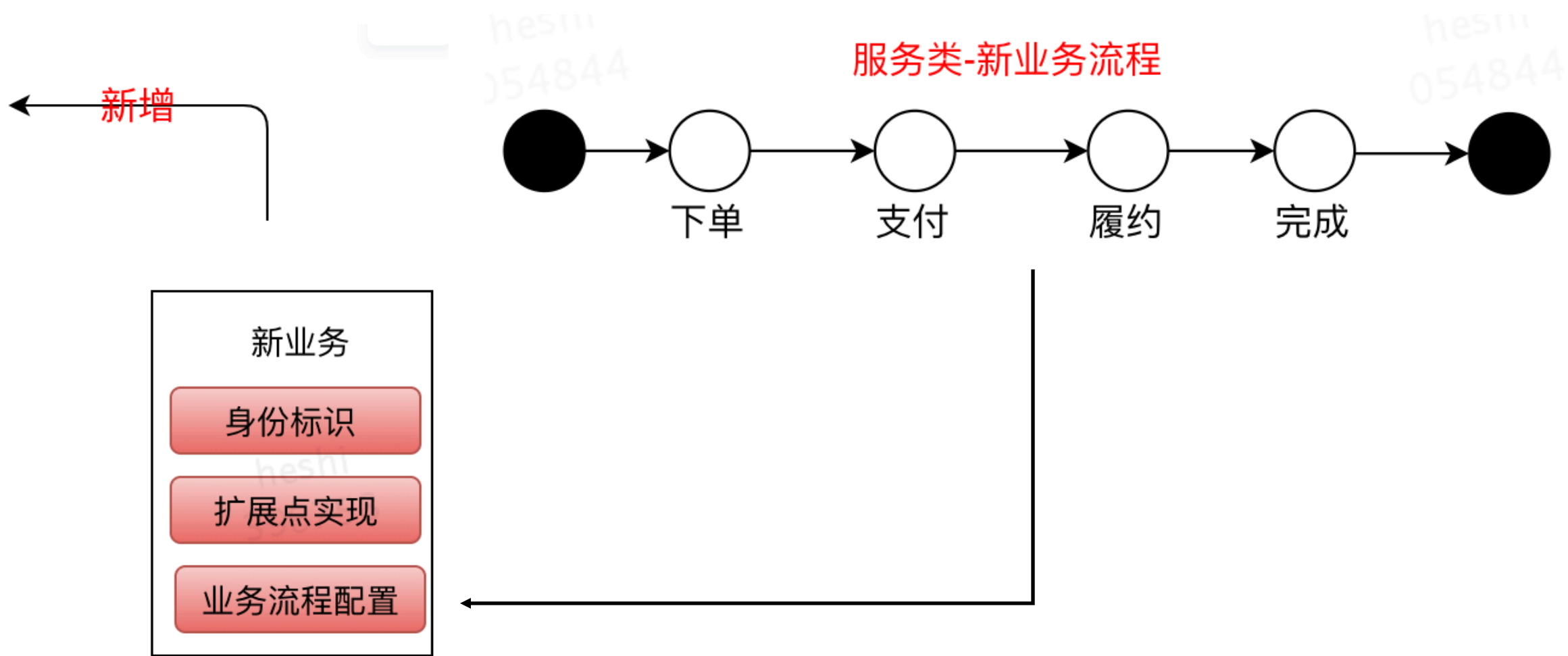
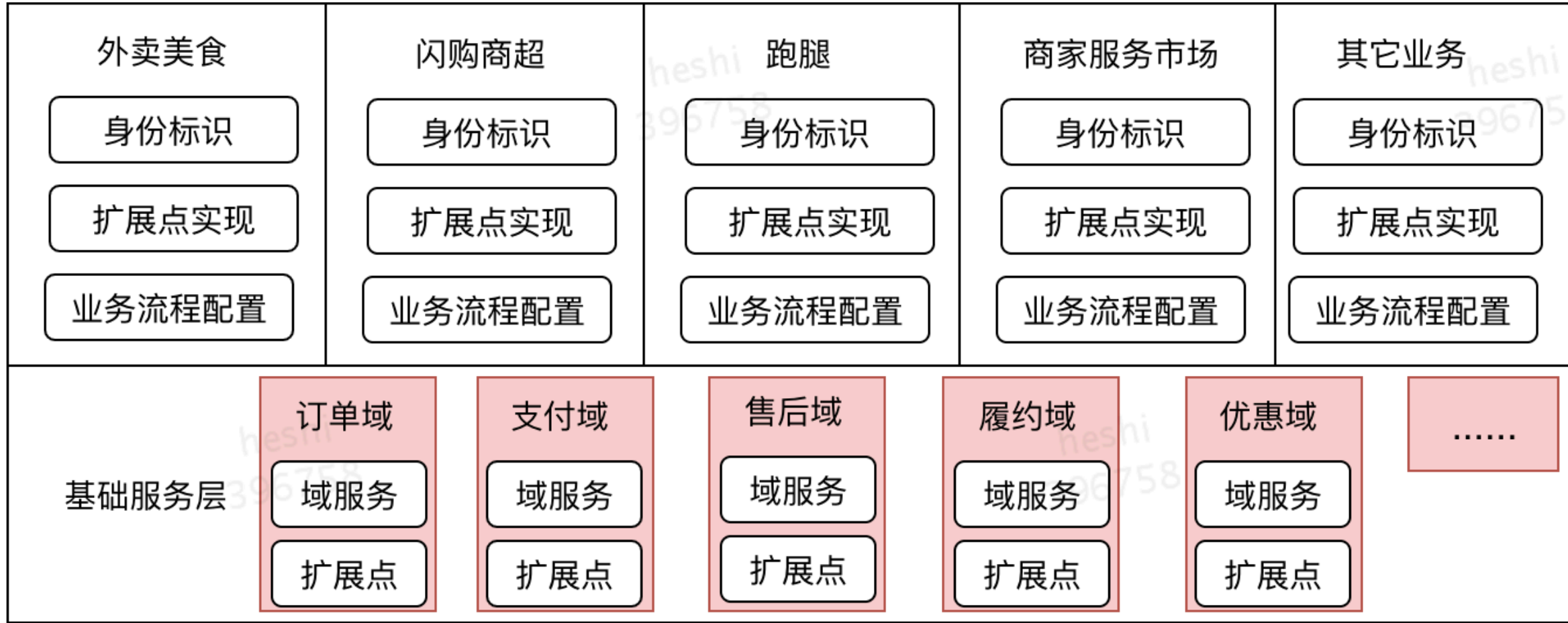
- ✓ 通用性&扩展性：
 - ✓ 业务身份定义：通过业务身份+业务标识定义一套灵活的业务身份定义。
 - ✓ 业务模型扩展：通过应用MAP+存储JSON结构可以灵活支持业务模型扩展。
 - ✓ 业务功能通用&扩展：通过功能抽象+扩展展预留解决了功能通用化+差异逻辑扩展的需求。
 - ✓ 业务流程扩展：通过流程引擎支持业务流程灵活扩展。
- ✓ 稳定性：通过业务隔离+多业务监控保障平台支撑多业务时的稳定性。

收益：构建一套通用、可配置、易扩展的订单交易平台，统一支持多业务、多品类的订单交易。新业务接入时，效率得到极大的提升，以最近接入的一个新业务为例，接入时间为3天。同时，沉淀了大量的通用订单交易能力。

新业务接入-示例

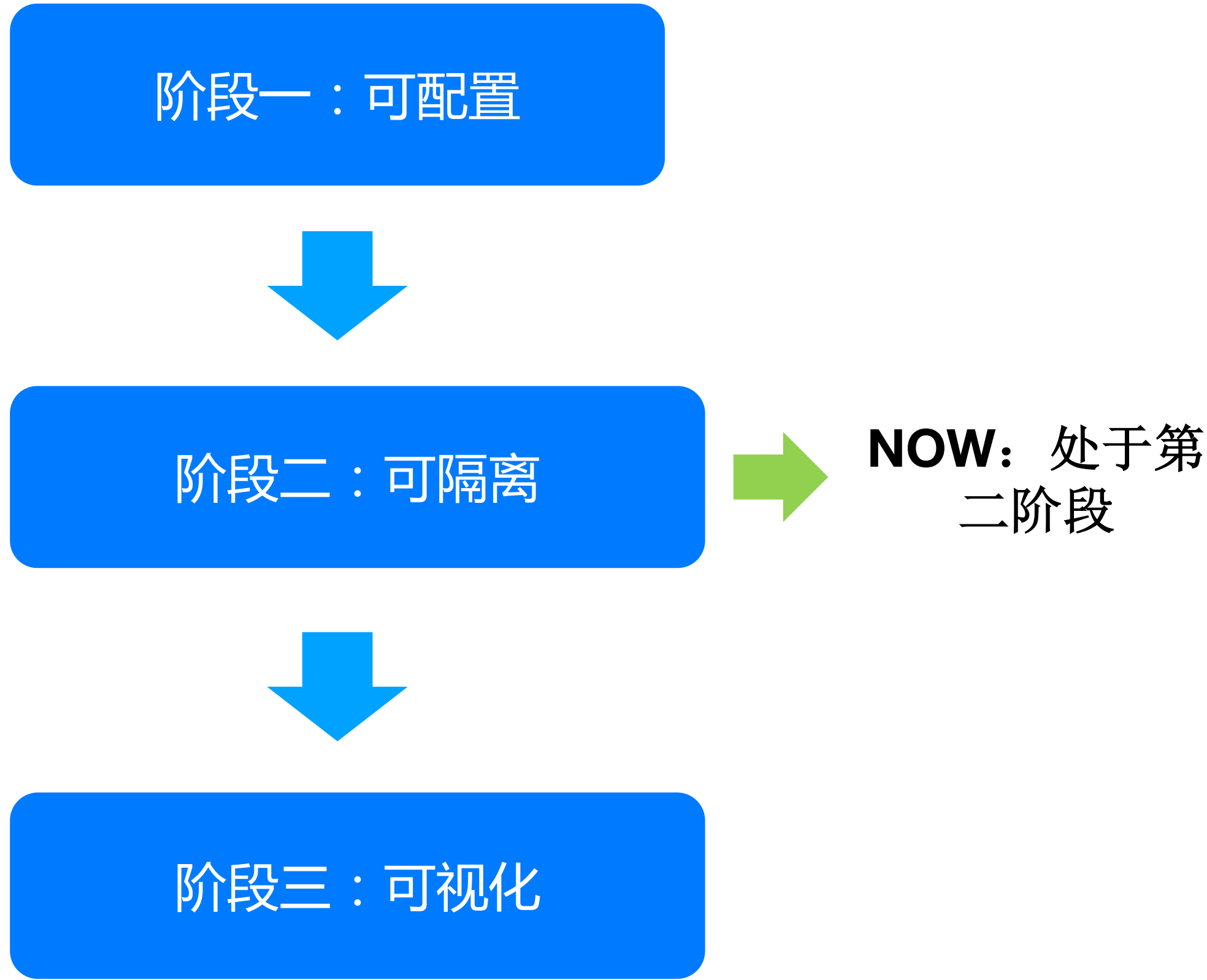
新业务接入流程：

- ✓ 增加业务身份标识；
- ✓ 根据业务需求，实现相应的扩展点；
- ✓ 根据业务需求，设计业务流程并编写XML；



后续规划：

- ✓ 能力透出&可视化：当前平台有哪些能力、支持哪些扩展都是记录在文档中，影响对接效率。后续计划以结构化、可视化的方式将这些能力及扩展点在后台展现，并支持在后台进行配置、调试、上线等，提升效率。





更多技术干货
欢迎关注“美团技术团队”

招聘：高级JAVA开发/架构师
邮箱：heshi@meituan.com

